TP Minmax

Licence Informatique - 3ème année

année 2023-2024

Le but de ce TP est d'implémenter différentes IA pour le jeu d'Othello, et de comparer leurs performances à la fois de jeu et en temps de calcul.

Code disponible

Celui ci est organisé en trois fichiers :

- jeulib, contenant les classes Jeu et Plateau, permettant d'interagir avec le jeu (compter les points, jouer un coup, obtenir la liste des coups disponibles...)
- interfacelib, contenant la classe Interface, qui génère un GUI avec lequel on peut interagir en cliquant
- joueurlib, contenant la classe Joueur et ses sous classes, Humain et IA.

Vous créerez de nouvelles IA en spécifiant leur méthode **demande_coup** qui devra renvoyer le coup joué par l'IA. Un coup est une liste [i,j] où i et j sont les coordonnées de la case jouée. La liste vide [] est un coup qui signifie que le joueur passe son tour.

1. Commencez par jouer une partie contre vous même pour vérifier que le jeu marche bien chez vous. Pour cela il suffit de créer une instance de la classe Jeu, et d'utiliser la méthode **demarrer**.

Random

2. Codez ensuite un Joueur Random, sélectionnant à chaque tour avec une probabilité uniforme un coup parmi ceux possibles. Vous pourrez utiliser la méthode **liste_coups_valides** de la classe Plateau. Une instance de IA peut accéder au plateau via son attribut jeu, qui lui même a un attribut plateau. Vous pourrez vous servir de la fonction **numpy.random.randint(a,b)** qui permet de tirer un entier entre a et $b-1$ inclus.

Minmax

3. Comme l'arbre de jeu est trop grand pour être entièrement évalué, créez une fonction d'évaluation d'une position (prenez quelque chose de simple pour commencer).
4. Implementez l'algorithme minmax en limitant la profondeur de l'exploration (vous pourrez prendre une profondeur 5, et augmenter en fonction des capacités de votre machine). Vous pourrez utiliser la méthode **copie** de la classe Plateau pour pouvoir explorer différents noeuds sans impacter la partie en cours.
5. Ajoutez un compteur du nombre de fois que votre Joueur utilise la méthode **jouer** (y compris pour l'exploration via minmax).
6. Ajoutez également un compteur de temps physique via la librairie time.
7. Quel intérêt voyez vous à avoir ces deux compteurs différents ? Comment évoluent-ils l'un par rapport à l'autre ?
8. Tracez une courbe montrant comment les deux compteurs varient en fonction du choix de la profondeur maximale.
9. Ajoutez une fonction permettant de simuler n parties entre deux IA, et donnant le nombre de victoires, score moyen ainsi que le temps moyen de calcul pour chaque camp.

Alpha-Beta

10. Créez un nouveau Joueur pour l'algorithme Alpha-Beta.

11. Sans faire l'expérience, quel sera à votre avis le résultat de 100 parties avec Minmax en joueur noir et Alpha-Beta en joueur blanc, et pourquoi ?

12. Comparez les performances de temps avec Minmax, en faisant varier la profondeur maximale.

13. Faites une nouvelle version d'Alpha-Beta, où depuis un noeud de l'arbre de jeu, le prochain choisi pour l'exploration est le noeud ayant la meilleure valeur selon votre fonction d'évaluation.

14. Comparez cette nouvelle version à l'ancienne.

15. Définissez une nouvelle fonction d'évaluation pour améliorer votre algorithme. Vous pouvez prendre juger que les coins sont mieux que les bords qui sont mieux que le reste, retirer du score aux cases permettant à l'adversaire d'accéder aux coins, considérer le nombre de coups possibles du joueur ou de son adversaire...

Partie Bonus : MCTS

Dans l'algorithme MCTS (Monte Carlo Tree Search), chaque coup possible est évalué via des simulations de parties aléatoires, appelées rollouts. On évaluera le résultat d'une simulation soit par le score obtenu, soit par 0 pour une défaite et 1 pour une victoire.

16. Faire une fonction **rollout**, prenant un Plateau et jouant des coups aléatoire sur une copie de ce Plateau jusqu'à la fin de la partie, puis retournant l'évaluation de fin de partie.

17. En déduire un premier joueur MCTS, qui possède un budget de N rollouts, qu'il dépensera également entre les différents coups possibles afin de les évaluer, et retournera le coup ayant la plus grande valeur moyenne.

Amélioration de MCTS avec UCB (Upper Confidence Bound): l'idée principale est qu'il n'est pas efficace de dépenser équitablement son budget entre les différents coups possibles: il y a un arbitrage à faire entre à quel point un coup semble mauvais (et donc donne peu envie de le reessayer), et à quel point ce coup a été peu testé (et donc que l'appréciation obtenue peut être due à de la malchance). Pour p un plateau, on selectionnera à la réévaluation le plateau enfant e maximisant

$$v(e) := \frac{w_e}{n_e} + c\sqrt{\frac{\log n_p}{n_e}} \ , \tag{1}$$

où n_b est le nombre de fois où un plateau b a été selectionné pour être évalué, w_e est le nombre de victoires où le score total obtenu en évaluant e (et donc w_e/n_e est la moyenne), et c est le paramètre d'exploration, qui peut être pris à $\sqrt{2}$. On pourra y voir le maximum d'un intervalle de confiance, dont la largeur est controlée par la variance de la moyenne, qui est inversement proportionnelle à la racine carrée du nombre d'échantillons, en accord avec le théorème de limite centrale.

18. Faire un joueur UCT (Upper Confidence Tree), implémentant cette combinaison de MCTS avec UCB.

19. Faites des comparaisons des performances entre Alpha-Beta, MCTS et UCT, en fonction d'un budget du nombre de coups simulés ou de temps de calcul alloué.